
ProjectName – C/Java/Python

Étudiant1, Étudiant2, Étudiant3, Étudiant4, Étudiant5

28 janvier 2026

Table des matières

Introduction

Ce rapport préliminaire expose les fondations théoriques et architecturales du projet Scrabble en Java. Nous présentons l'historique du jeu, sa nature combinatoire, les défis algorithmiques qu'il pose, puis détaillons nos choix d'architecture et de structures de données.

1 Historique et Nature du Sujet

1.1 Historique du Scrabble

En 1931, l'architecte au chômage Alfred Mosher Butts décide de créer un jeu qui combine la chance du tirage au sort et la réflexion. En analysant la fréquence des lettres à la une du *New York Times*, il attribue à chaque jeton une valeur spécifique. Initialement appelé **Lexiko**, puis **Criss-Cross Words**, le jeu est rejeté par de nombreux fabricants avant qu'un homme d'affaires, James Brunot, ne rachète les droits en 1948 pour lui donner son nom définitif : **Scrabble**.

Le succès mondial du jeu tient d'un véritable coup de chance marketing. Dans les années 1950, Jack Straus, président de Macy's (un célèbre grand magasin new-yorkais), découvre le jeu pendant ses vacances et s'étonne de ne pas le trouver dans ses rayons. Il passe alors une commande massive, propulsant le Scrabble au rang de phénomène culturel. Aujourd'hui, avec plus de 150 millions d'exemplaires vendus et des compétitions internationales acharnées, il est devenu le jeu de lettres le plus populaire de la planète.

Dès les années 1980 et 1990, les premiers logiciels de jeu contre l'ordinateur ont vu le jour, suivis par l'explosion des versions en ligne comme l'Internet Scrabble Club (ISC) ou l'application mobile *Scrabble Go*. Au-delà du simple divertissement, l'informatique a révolutionné la compétition de haut niveau grâce à des logiciels d'analyse ultra-puissants comme **Quackle**. Le Scrabble est aujourd'hui un jeu qui a été codé des centaines de fois ; de nombreux projets *open source* sont disponibles en ligne et de nombreuses applications existent pour tous les appareils.

1.2 Nature et Enjeux du Jeu

Le Scrabble est un jeu de lettres à **information incomplète** (on ignore le tirage adverse) et à forte composante stratégique. L'objectif est de maximiser un score sur une grille de 15×15 cases (21×21 en mode Super Scrabble) en plaçant des mots issus d'un dictionnaire défini au préalable, pouvant changer notamment en fonction de la langue.

Le Scrabble impose la maîtrise de trois dimensions interdépendantes :

1. **Logique spatiale** : Le placement des lettres doit respecter une règle de connexité stricte (tout mot posé doit être rattaché au jeu) et une vérification d'orthogonalité (chaque mot formé par collision doit être valide).
2. **Logique mathématique** : Le calcul des points n'est pas une simple somme. Il s'agit d'un système de pondération dynamique où il faut prendre en compte la valeur de chaque lettre posée et/ou récupérée, les bonus s'appliquant respectivement aux lettres ou aux mots (lettre compte double, lettre compte triple, mot compte double, mot compte triple), l'ordre d'application des bonus (lettres puis mots), ainsi que la gestion des *Scrabble* (+50 points si l'ensemble des 7 lettres est posé).
3. **Logique linguistique** : Le programme doit arbitrer instantanément la validité de centaines de milliers de mots.

1.3 Problématique Algorithmique

Le cœur du sujet réside dans la gestion de l'explosion combinatoire. Avec un dictionnaire pouvant comptabiliser des dizaines voire centaines de milliers de mots (380 000 pour celui de la langue française) et un chevalet de 7 lettres, le nombre de placements théoriques sur une grille est astronomique.

L'explication du sujet porte donc sur la mise en place d'un moteur capable de :

1. **Rechercher exhaustivement** tous les coups légaux possibles sur une grille donnée.

2. **Indexer le lexique** de manière à ce que le temps de réponse soit imperceptible pour l'utilisateur.
3. **Simuler une intelligence artificielle** capable de prendre des décisions non seulement basées sur le score du mot, mais aussi sur les points récupérés par le placement de ce mot sur le plateau, l'obstruction apportée par celui-ci, ainsi que l'état après ce coup (accessibilité aux bonus, lettres restantes dans le rack, etc.).

2 Algorithmes et Structures de Données

L'implémentation d'un joueur artificiel performant au Scrabble repose sur la résolution de deux problèmes distincts : la génération efficace de tous les coups légaux et la sélection du meilleur coup parmi ceux-ci. Cette section détaille les choix algorithmiques retenus pour répondre à ces besoins.

2.1 Modélisation du problème

Contrairement aux Échecs ou au Go, qui sont des jeux à information parfaite et déterministes, le Scrabble est un jeu à information imparfaite, on ne connaît pas lettres de l'adversaire et le tirage des lettres repose sur le hasard.

Une approche gloutonne, qui consisterait à jouer systématiquement le coup rapportant le plus de points à l'instant t , est sous-optimale. Elle néglige deux facteurs cruciaux :

- **Le reliquat (Rack Leave)** : La qualité des lettres conservées pour le tour suivant.
- **L'ouverture du plateau** : Le risque d'offrir une case "Mot Compte Triple" à l'adversaire.

Pour pallier ces manques, nous avons deux algorithmes de théorie des jeux : Minimax et Expectiminimax.

2.2 L'Algorithme Minimax

L'algorithme Minimax est la base de la décision dans les jeux à somme nulle. Il parcourt l'arbre de jeu jusqu'à une profondeur d donnée.

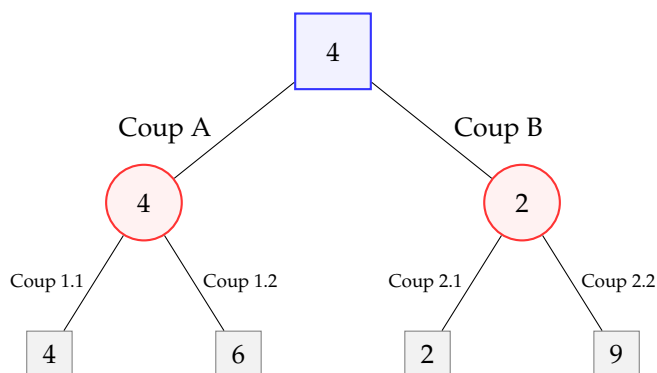


FIGURE 1 – Exemple de déroulement d'un arbre Minimax

Fonctionnement

L'algorithme alterne entre deux types de nœuds :

- **Nœuds MAX** : Cherche à maximiser le score final.
- **Nœuds MIN** : Cherche à minimiser le score.

L'application stricte de Minimax au Scrabble pose un problème de modélisation. Minimax suppose que l'adversaire jouera toujours le coup optimal pour nous contrer. Or, pour déterminer ce coup optimal, l'algorithme doit connaître les lettres de l'adversaire. Dans une simulation Minimax, nous devrions supposons le pire cas : l'adversaire possède exactement les lettres nécessaires pour réfuter notre coup (ex : un "Z" pour atteindre un mot compte triple ouvert). Cette approche rend l'IA excessivement défensive et inadaptée à la réalité probabiliste du tirage.

Listing 1: Pseudo-code de l'algorithme Minimax

2.3 L'Algorithme Expectiminimax

Pour intégrer la dimension aléatoire du tirage de lettres, nous utilisons l'algorithme **Expectiminimax**. C'est une variation du Minimax qui introduit un troisième type de nœud : les **Nœuds de Hasard**.

L'arbre de décision se décompose désormais en trois niveaux par tour de jeu :

1. **Nœud MAX** : L'IA choisit un coup parmi les coups possibles générés.

2. **Nœud CHANCE** : Ce nœud représente l'incertitude sur les lettres de l'adversaire. Il possède une branche pour chaque tirage possible, pondérée par sa probabilité.
3. **Nœud MIN** : L'adversaire joue le meilleur coup possible avec le tirage attribué par le nœud chance.

La valeur d'un nœud de hasard n'est ni un minimum ni un maximum, mais une espérance mathématique. Pour un état s correspondant à un nœud de hasard :

$$\text{Expect}(s) = \sum_{r \in \text{Tirages}} P(r) \times v(\text{successeur}(s, r))$$

Où $P(r)$ est la probabilité que le tirage soit r , sachant les lettres déjà visibles.

Listing 2: Pseudo-code de l'algorithme Expectiminimax

Défi combinatoire et Échantillonnage

Le facteur de branchement aux nœuds de hasard est gigantesque. Simuler tous les tirages possibles de 7 lettres parmi les N lettres restantes est impossible en temps réel.

Pour contourner cette complexité, nous utiliserons une méthode d'approximation par échantillonnage. Au lieu de calculer la somme sur tous les tirages, nous générons un sous-ensemble représentatif de N mains adverses aléatoires (ex : 20 simulations) basées sur la distribution réelle des lettres restantes.

L'algorithme devient alors :

1. L'IA joue un coup C .
2. Nous simulons N mains adverses possibles.
3. Pour chaque main, nous calculons la meilleure réponse de l'adversaire.
4. La valeur du coup C est la moyenne des scores résultant de ces simulations.

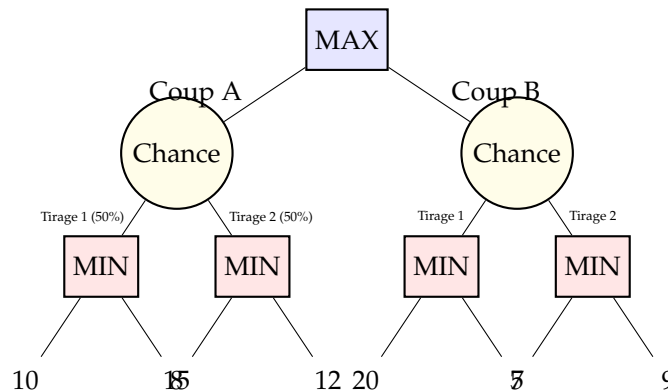


FIGURE 2 – Représentation d'un arbre Expectiminimax à 3 niveaux

3 Structure de données pour la recherche : GADDAG

Pour que les algorithmes de décision (Minimax ou Expectiminimax) soient réellement performants, ils doivent pouvoir accéder à la liste exhaustive des coups légaux de manière quasi instantanée. À cette fin, nous utilisons la structure de données **GADDAG**, inventée par Steven Gordon en 1994.

Inspiré du **DAWG** (Directed Acyclic Word Graph), le GADDAG est spécifiquement optimisé pour le Scrabble, car il facilite la construction de mots autour d'une "ancre" (une lettre déjà présente sur le plateau).

3.1 Fonctionnement et Transformation

Le principe du GADDAG repose sur une transformation du dictionnaire. Pour chaque mot, plusieurs représentations sont générées en le découpant autour de chaque lettre ancre. La partie située à gauche de cette ancre est inversée, puis séparée de la partie droite par un caractère spécial (souvent $>$ ou $\$$), puis la fin du mot est notée normalement.

Par exemple, pour le mot **"CHAT"**, les représentations stockées dans l'arbre sont les suivantes :

- C > H A T (Ici, l'ancree est C)
- H C > A T (Ici, l'ancree est le H, donc on met un C à sa gauche, puis on passe à droite pour poser A-T)

- A H C > T (Ici, l'ancre est le A, donc on met un C et un H à sa gauche, puis on passe à droite pour poser le T)
- T A H C > (Ici, l'ancre est le T, et on complète tout le mot vers la gauche : A, puis H, puis C)

Cette redondance permet à l'algorithme de trouver des mots quel que soit le point d'ancrage sur la grille, sans avoir à "deviner" le début du mot.

3.2 Comparaison : GADDAG vs DAWG

Contrairement au GADDAG, l'algorithme **DAWG** ne fonctionne que de manière unidirectionnelle (de la première à la dernière lettre).

Critère	DAWG	GADDAG
Vitesse	Plus lent (doit deviner l'origine du mot).	Très rapide (accès direct via n'importe quelle lettre).
Mémoire	Très compact.	Plus lourd (5 fois la taille d'un DAWG en moyenne).
Complexité	Plus simple à implémenter.	Plus complexe et structure plus dense.

TABLE 1 – Comparaison des structures de données pour le Scrabble

De plus, il y a un moyen montrer dans l'article original publiant l'idée du GADDAG, de réduire la mémoire utilisée par le GADDAG. Le moyen utilisé est de réduire le nombre de sorties pour chaque mot en reliant les répétitions inutiles de fin de mots comme le montre cet exemple fait pour le mot CARE.

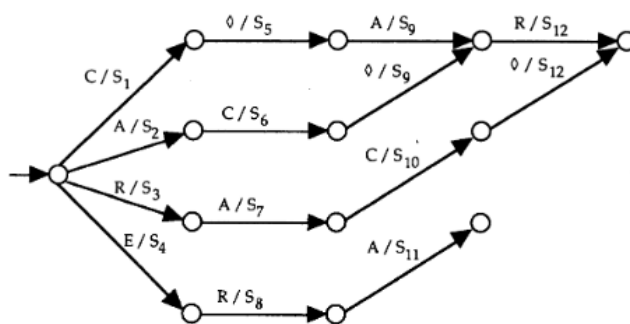


FIGURE 3 – Graphe du GADDAG semi-minimisé pour « CARE ». (Source : [2])

3.3 Synthèse des performances

L'utilisation du GADDAG présente des avantages majeurs pour notre moteur de jeu :

- **Efficacité maximale** : C'est l'algorithme le plus rapide pour la génération de coups, évitant les répétitions inutiles.
- **Polyvalence d'ancrage** : Il permet de trouver efficacement des mots s'insérant entre deux lettres ou se collant parallèlement à un mot existant.
- **Unicité du chemin** : Une fois la structure construite, il n'existe qu'un seul chemin pour chaque mot avec un point d'ancrage donné.

En résumé, bien que le GADDAG soit plus exigeant en termes de mémoire vive et plus complexe à coder, il constitue le meilleur choix pour garantir une IA capable de rivaliser avec des joueurs experts en analysant des milliers de combinaisons par seconde.

4 Spécifications étendues

F16 : Proposer de sauvegarder avant de quitter

Description : Le programme doit demander à l'utilisateur s'il souhaite sauvegarder la partie avant de quitter, si des modifications non sauvegardées existent.

Prérequis :

- F2 : Fichier de configuration (pour définir les chemins de sauvegarde par défaut).
- F21 : Format de sauvegarde (pour savoir comment structurer les fichiers de sauvegarde).
- F15 : Shell interactif (pour gérer les commandes comme quit et save).

Type de besoin :

Fonctionnel.

Découpage en sous-besoins :

Pour implémenter cette fonctionnalité, nous avons identifié quatre sous-besoins principaux.

Sous besoins 1. Détecter les modifications non sauvegardées

Description : Le programme doit détecter si l'état actuel du jeu diffère de l'état sauvegardé. Cela inclut les modifications du plateau, des scores et des lettres restantes.

Actions :

- Stocker l'état du jeu après chaque sauvegarde.
- Comparer l'état actuel avec l'état sauvegardé à chaque tentative de quitter le programme.

Fonction associée :

```
1 boolean modificationsNonSauvegardees(EtatJeu etatActuel, EtatJeu etatSauvegarde)
```

Paramètres :

- `etatActuel` : Représente l'état actuel du jeu (plateau, scores, lettres restantes).
- `etatSauvegarde` : Représente l'état du jeu lors de la dernière sauvegarde.

Retour :

- `true` : S'il y a des modifications non sauvegardées.
- `false` : Si aucun changement n'a été fait depuis la dernière sauvegarde.

Test unitaire :

- **Cas 1 :** `etatActuel` contient le mot "CHAT" sur le plateau, `etatSauvegarde` est vide. Résultat attendu : `true` (car il y a des modifications).
- **Cas 2 :** `etatActuel` et `etatSauvegarde` sont identiques. Résultat attendu : `false` (pas de modifications).

Sous besoins 2. Demander confirmation à l'utilisateur

Description : Le programme doit afficher un message de confirmation et lire la réponse de l'utilisateur.

Actions :

- Afficher le message : Save the game before quitting? [y/N].
- Lire la réponse de l'utilisateur.
- Retourner `true` si la réponse est "y" ou "Y", `false` sinon.

Fonction associée :

```
1 boolean demanderConfirmation()
```

Retour :

- `true` : Si l'utilisateur répond "y" ou "Y".
- `false` : Si l'utilisateur répond "n", "N" ou toute autre réponse.

Test unitaire :

- **Cas 1 :** L'utilisateur répond "y". Résultat attendu : `true`.
- **Cas 2 :** L'utilisateur répond "n". Résultat attendu : `false`.

Sous besoins 3. Sauvegarder la partie

Description : Si l'utilisateur choisit de sauvegarder, le programme doit écrire l'état actuel du jeu dans un fichier.

Actions :

- Demander à l'utilisateur le chemin du fichier de sauvegarde.
- Écrire l'état du jeu dans le fichier au format défini (JSON ou format personnalisé).

Fonction associée :

```
1 boolean sauvegarderPartie(EtatJeu etatJeu, String cheminFichier)
```

Paramètres :

- `etatJeu` : L'état actuel du jeu à sauvegarder.
- `cheminFichier` : Le chemin du fichier où sauvegarder la partie.

Retour :

- `true` : Si la sauvegarde a réussi.
- `false` : Si une erreur survient.

Test unitaire :

- **Cas 1** : Sauvegarder un état de jeu valide dans un fichier accessible. Résultat attendu : `true` et le fichier contient les données correctes.
- **Cas 2** : Sauvegarder dans un chemin invalide (ex. : `/dossier/inexistant/fichier.json`). Résultat attendu : `false`.

Sous besoins 4. Gérer les erreurs de sauvegarde

Description : Le programme doit gérer les erreurs lors de la sauvegarde (ex. : permissions insuffisantes, disque plein) et redemander à l'utilisateur s'il souhaite réessayer.

Actions :

- Capturer les exceptions lors de l'écriture du fichier.
- Afficher un message d'erreur et redemander à l'utilisateur s'il veut réessayer.

Fonction associée :

```
1 boolean gererErreurSauvegarde(EtatJeu etatJeu)
```

Paramètres :

- `etatJeu` : L'état actuel du jeu à sauvegarder.

Retour :

- `true` : Si la sauvegarde a finalement réussi.
- `false` : Si l'utilisateur abandonne après une erreur.

Test unitaire :

- **Cas 1** : L'utilisateur entre un chemin invalide, puis un chemin valide. Résultat attendu : `true` après la deuxième tentative.
- **Cas 2** : L'utilisateur abandonne après une erreur. Résultat attendu : `false`.

5 Architecture du projet

L'objectif de ce projet est de concevoir une application robuste du Scrabble, capable de gérer aussi bien une interface console (CLI) qu'une interface graphique (JavaFX), tout en intégrant des fonctionnalités avancées comme le jeu en réseau et l'intelligence artificielle. Le projet est actuellement structuré sous forme de squelettes de classes.

5.1 Organisation Globale et MVC

Pour répondre à l'exigence de modularité (F7, F39), nous avons adopté le patron **MVC (Modèle-Vue-Contrôleur)**. Cette séparation est indispensable pour isoler la logique métier des différentes interfaces. L'implémentation s'appuie

sur le patron Observer pour notifier automatiquement les interfaces de toute modification du modèle, garantissant une synchronisation réactive tout en maintenant un couplage faible.

- **Le Modèle (`fr.u_bordeaux.scrabble.model`)** : Il contient l'état du jeu (plateau, sac, joueurs), ainsi que les packages `network`, `ia` et `dictionary`. Il est conçu pour être indépendant de l'affichage.
- **La Vue (`fr.u_bordeaux.scrabble.view`)** : Elle est divisée entre une implémentation `cli` (shell interactif avec codes ANSI pour les couleurs du plateau) et une implémentation `gui` sous JavaFX.
- **Le Contrôleur (`fr.u_bordeaux.scrabble.controller`)** : Il gère les entrées utilisateur et les options de lancement via `commons-cli`. Il assure également le chargement de la configuration `.scrabblerc` (format INI).

5.2 Architecture du Cœur de Jeu et Flexibilité

L'architecture que nous visons comporte une séparation nette entre les données brutes et les règles de gestion, permettant au logiciel de s'adapter dynamiquement aux variantes du jeu et aux environnements linguistiques.

- **Plateau Dynamique** : La classe `Board` utilise une matrice d'objets `Square`, chacun associé à un `SquareType` (énumération gérant les bonus de score). Cette structure permet de passer d'un plateau standard 15×15 au mode Super-Scrabble 21×21 sans modifier la logique de placement. Les modules du package `factory` implémenteront le design pattern **Factory** pour générer ces différents plateaux, dans le but de séparer l'instanciation des grilles de leur utilisation par le moteur de jeu (et de faciliter l'ajout de nouvelles variantes sans modifier le code existant). »
- **Abstraction des Joueurs** : Nous utilisons une classe abstraite `Player` pour traiter de manière uniforme les joueurs humains (locaux), les intelligences artificielles et les joueurs réseaux. Chaque joueur possède son propre `Rack` (chevalet) de 7 objets `Tiles` piochés dans `Bag`, qui pourront être joués sur le `Board` avec un `Move`.
- **Internationalisation et Localisation (i18n/l10n)** : L'application utilise les mécanismes `ResourceBundle` et `Locale` de Java pour supporter l'anglais et le français (choisi automatiquement en fonction des variables d'environnement `LANG` ou `LC_ALL`). Cette gestion impacte directement la logique du jeu en chargeant dynamiquement la distribution des lettres, leurs valeurs respectives dans le sac (`Bag`), ainsi que le dictionnaire associé via le module `lexicon`.

5.3 Gestion des Coups et Persistance

La gestion des actions repose sur une traçabilité complète, nécessaire pour les fonctionnalités de confort utilisateur et la sauvegarde.

- **Pattern Command** : Chaque action est encapsulée dans un objet `Move`, avec le choix de l'action spécifié par un enum `MoveType` (`PLAY`, `EXCHANGE` et `PASS`). La logique d'application des `Move` sera géré par la classe `MoveHandler`, pour séparer cette logique des données. Ces choix d'implémentation faciliteront l'implémentation d'undo/redo, des fichiers de sauvegarde et des différents joueurs, car chaque coup devient une action indépendante qu'on peut simplement empiler, annuler ou stocker sans toucher au reste du code.
- **Format de Sauvegarde Robuste** : Les fichiers de sauvegarde (`.txt` en ASCII) sont structurés en trois blocs obligatoires : `[settings]`, `[game]` et `[history]`. Notre parseur sera robuste en ignorant les commentaires en ligne (`#`) ou en bloc (`{...}`) et en signalant précisément la ligne en cas d'erreur de format.

5.4 Algorithmes, Dictionnaire et Réseau

Cette section regroupe les composants les plus techniques du projet, où la performance du moteur de jeu rencontre les capacités d'extension réseau.

- **Moteur de recherche et Dictionnaire** : La validation des mots et la recherche de coups reposent sur le module `dictionary`. Il s'agit d'une interface abstraite permettant d'interchanger deux structures de données haute performance : le **DAWG** (F28), optimisé pour la vérification rapide, et le **GADDAG** (F29), indispensable pour générer efficacement tous les coups possibles à partir d'un tirage. Le module `lexicon` assure le chargement de ces données en fonction de la langue choisie (F26).
- **Intelligence Artificielle** : L'IA exploite les capacités du **GADDAG** pour trouver les meilleurs placements et simuler des joueurs artificiels `AIPlayer`. Au-delà d'une simple recherche de score maximal, l'activation de l'option `-ai-exptiminimax` lance un algorithme qui anticipe les futurs tirages et les réponses probables de l'adversaire (F36). Pour la recherche de mots expérimentale, nous prévoyons l'intégration de la bibliothèque **TensorFlow Java API** (F37) avec un temps de réflexion borné à 5 secondes par défaut.

- **Architecture Réseau** : L'infrastructure est centralisée par un `NetworkManager` qui orchestre les échanges de données. Le `GameServer` gère les connexions multi-threadées via TCP sur le port 12345, tandis que le `GameClient` assure la liaison avec le moteur local. Les interactions (coups, invitations) sont encapsulées dans des objets `Packet` transitant au format ASCII. La découverte automatique s'appuie sur un broadcast UDP sur le port 12346, avec un système d'invitations expirant après 5 minutes (F40).

5.5 Environnement et Qualité

Le projet est automatisé via **Maven** pour un déploiement sur **GNU/Linux**. La fiabilité est garantie par une suite de tests **JUnit** associée à **Jacoco**, avec un objectif de **85% de couverture**. L'intégralité du code et de la **Javadoc** respecte les standards **Google** en anglais. Enfin, la robustesse est assurée par un traitement systématique des erreurs sur **stderr**, garantissant une exécution sans crash.

Références

- [1] Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014.
- [2] Steven A. Gordon. A faster scrabble move generation algorithm. *Softw. Pract. Exp.*, 24(2) :219–232, 1994.